

**LAPORAN PROJECT EVALUASI TENGAH
SEMESTER (ETS)
ALGORITMA DAN PEMROGRAMAN
PROJECT BASED LEARNING (PBL)**

**Sistem Monitoring dan Kontrol Irigasi Otomatis Berbasis
Rust**

Dosen Pengampu:
Ahmad Radhy, S.Si., M.Si.

Disusun Oleh:
Arina Sofa'il Assoriyyah
NRP: 2042251022
Emil Juan Paskah Purba
NRP: 2042251039

**PROGRAM STUDI D4 TEKNOLOGI REKAYASA
INSTRUMENTASI
DEPARTEMEN TEKNIK INSTRUMENTASI
INSTITUT TEKNOLOGI SEPULUH NOPEMBER
SURABAYA
2026**

Contents

1	Pendahuluan	2
2	Studi Kasus Instrumentasi	2
2.1	Konfigurasi Sensor (Pengukuran Spasial)	2
2.2	Logika Pengendalian (<i>Controller</i>)	3
3	Algoritma dan Flowchart	3
4	Penjelasan Program Rust	4
4.1	Fitur Validasi Input (<i>Robustness</i>)	4
4.2	Struktur Data dan Objek	4
4.3	Mekanisme Perulangan (<i>Looping</i>)	4
5	Konsep OOP	5
5.1	Enkapsulasi Data dengan <code>struct</code>	5
5.2	Abstraksi Perilaku dengan <code>impl</code>	5
6	Implementasi Komputasi Numerik	5
6.1	Formulasi Matematis	6
6.2	Integrasi pada Algoritma	6
7	Hasil Pengujian	6
7.1	Skenario 1: Kondisi Genangan Ideal	6
7.2	Skenario 2: Kondisi Air Surut, Tanah Basah	7
7.3	Skenario 3: Kondisi Kekeringan Ekstrem	7
8	Analisis Hasil	8
8.1	Kesesuaian Logika <i>Controller</i> dan Efisiensi	8
8.2	Keandalan Pengolahan Data (<i>Spatial Average</i>)	8
8.3	Ketahanan Program (<i>Robustness</i>)	8
9	Kesimpulan	9

1 Pendahuluan

Manajemen air pada lahan persawahan merupakan faktor krusial yang menentukan produktivitas dan keberhasilan panen padi. Berbeda dengan tanaman perkebunan lainnya, padi membutuhkan kontrol genangan air (level) dan kelembapan tanah yang presisi, terutama pada fase pertumbuhan vegetatif. Pengontrolan irigasi dan pemantauan kondisi lahan secara manual sering kali tidak efisien, padat karya, dan rentan terhadap keterlambatan penanganan yang dapat berujung pada kekeringan ekstrem atau pemborosan sumber daya air.

Oleh karena itu, diperlukan sebuah sistem instrumentasi tertutup (*closed-loop control system*) yang mampu memonitor parameter fisik lahan secara *real-time* dan mengendalikan aktuator secara otomatis.

Project pada Evaluasi Tengah Semester (ETS) ini bertujuan untuk merancang dan mengembangkan purwarupa perangkat lunak (*software*) dari sistem kontrol irigasi otomatis tersebut menggunakan bahasa pemrograman Rust. Sistem ini disimulasikan sebagai aplikasi berbasis terminal (*console application*) yang membaca input multi-variabel, yaitu level air (cm) dan kelembapan tanah (%) dari tiga titik sensor yang berbeda.

Dengan mengintegrasikan konsep Pemrograman Berbasis Objek (OOP) serta komputasi numerik untuk perhitungan rata-rata spasial (*spatial average*), program ini dirancang untuk bertindak sebagai *controller* cerdas. *Controller* akan mengevaluasi data sensor dan mengambil keputusan berjenjang untuk mengaktifkan atau menonaktifkan pompa irigasi sesuai dengan batas aman (*set point*) agrikultur yang telah ditentukan.

2 Studi Kasus Instrumentasi

Studi kasus yang diangkat pada evaluasi ini adalah **Sistem Kontrol Irigasi Otomatis pada Lahan Persawahan Berbasis Multi-Variabel**[cite: 33, 44]. Persawahan merupakan *plant* (objek kendali) yang memiliki karakteristik dinamika sistem yang unik, di mana ketinggian genangan air merupakan prioritas utama dibandingkan sekadar kelembapan tanah.

2.1 Konfigurasi Sensor (Pengukuran Spasial)

Untuk mendapatkan representasi data yang akurat dan menghindari pembacaan yang bias akibat kontur tanah yang tidak rata, pengukuran tidak dilakukan pada satu titik tunggal. Sistem ini mensimulasikan pembacaan dari enam buah sensor yang tersebar di tiga titik spasial yang berbeda (misalnya: dekat pintu masuk air, di tengah lahan, dan di ujung saluran terjauh). Pada masing-masing titik, sistem memantau dua parameter:

- **Sensor Ultrasonik (Level Air - cm):** Mengukur ketinggian genangan air di atas permukaan tanah.
- **Sensor *Soil Moisture* (Kelembapan - %):** Mengukur persentase kadar air di dalam tanah.

2.2 Logika Pengendalian (*Controller*)

Sistem menggunakan strategi kendali lingkaran tertutup (*closed-loop control*) dengan metode aksi *On-Off* berjenjang. Nilai batas (*set point*) yang ditetapkan sebagai acuan *controller* adalah batas genangan ideal (3 cm), batas genangan kritis (1 cm), dan batas kelembapan kritis (65%). Aktuator berupa pompa irigasi air akan dikendalikan berdasarkan logika prioritas berikut:

1. **Prioritas 1 (Aman):** Jika rata-rata genangan air ≥ 3 cm, tanah dipastikan basah dan kebutuhan air padi terpenuhi. Pompa dimatikan (*OFF*).
2. **Prioritas 2 (Siaga):** Jika air surut (< 3 cm) namun tanah di bawahnya masih menahan kelembapan yang baik ($\geq 65\%$), aktuator tidak perlu membuang energi. Pompa dalam status *Standby*.
3. **Prioritas 3 (Kritis):** Jika genangan hampir habis (< 1 cm) dan tanah mulai mengering secara ekstrem ($< 65\%$), pompa langsung diaktifkan secara maksimal (*ON*) untuk mencegah tanaman padi mengalami *water stress*.

3 Algoritma dan Flowchart

Sistem monitoring dan kontrol irigasi ini dirancang menggunakan algoritma *closed-loop* yang berjalan secara terus-menerus (*infinite loop*) selama sistem diaktifkan...

Alur pengambilan keputusan sistem kontrol tersebut divisualisasikan secara komprehensif pada Gambar 1 di bawah ini:

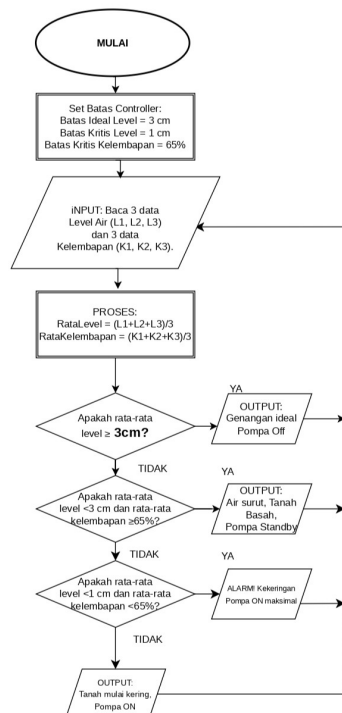


Figure 1: Flowchart Sistem Kontrol Irigasi Berbasis Multi-Variabel

4 Penjelasan Program Rust

Implementasi perangkat lunak pada proyek ini menggunakan bahasa pemrograman Rust untuk menjamin keamanan memori (*memory safety*) dan performansi yang optimal. Program dikembangkan dengan pendekatan modular yang memisahkan antara fungsi pembantu, struktur data, dan logika utama.

4.1 Fitur Validasi Input (*Robustness*)

Salah satu aspek penting dalam sistem instrumentasi adalah ketahanan terhadap data yang korup atau kesalahan input pengguna. Program ini dilengkapi dengan fungsi `baca_input` yang memanfaatkan fitur *pattern matching* (`match`) untuk memastikan hanya data bertipe numerik (`f32`) yang diproses. Jika pengguna memasukkan karakter non-numerik, program tidak akan *crash*, melainkan memberikan peringatan dan meminta input ulang.

```
1 fn baca_input(prompt: &str) -> f32 {
2     loop {
3         print!("{}", prompt);
4         io::stdout().flush().unwrap();
5         let mut input = String::new();
6         io::stdin().read_line(&mut input).expect("Gagal membaca
input");
7         match input.trim().parse::<f32>() {
8             Ok(angka) => return angka,
9             Err(_) => println!("ERROR: Input tidak valid!"),
10        }
11    }
12 }
```

Listing 1: Implementasi Validasi Input

4.2 Struktur Data dan Objek

Program menggunakan dua struktur utama (*struct*) untuk mengelola data secara terorganisir:

- **LahanSawah:** Menyimpan enam data mentah dari sensor (tiga titik level air dan tiga titik kelembapan).
- **ControllerIrigasi:** Menyimpan parameter *set point* yang menjadi acuan keputusan sistem.

4.3 Mekanisme Perulangan (*Looping*)

Sesuai dengan karakteristik sistem monitoring, program utama dijalankan di dalam blok `loop`. Hal ini memungkinkan sistem untuk mensimulasikan pemantauan kondisi lahan secara terus-menerus. Setiap siklus perulangan merepresentasikan satu unit waktu pengambilan data (*sampling time*), di mana sistem akan meminta input baru, melakukan komputasi, dan memperbarui status aktuator.

5 Konsep OOP

Program ini mengimplementasikan paradigma Pemrograman Berbasis Objek (*Object-Oriented Programming*) menggunakan konstruksi `struct` dan `impl` pada bahasa Rust. Pendekatan ini memungkinkan pemetaan aset fisik di lapangan ke dalam representasi digital yang terorganisir.

5.1 Enkapsulasi Data dengan struct

Dalam sistem ini, terdapat dua objek utama yang didefinisikan untuk memisahkan tanggung jawab data:

1. **struct LahanSawah:** Bertindak sebagai representasi fisik dari lahan yang dimonitor. Atribut di dalamnya mencakup enam data sensor yang mewakili kondisi *real-time* dari variabel proses (Level Air dan Kelembapan).
2. **struct ControllerIrigasi:** Bertindak sebagai otak atau unit kendali. Atribut di dalamnya menyimpan nilai *Set Point* atau ambang batas (*threshold*) yang menjadi acuan logika kontrol.

5.2 Abstraksi Perilaku dengan impl

Perilaku atau fungsi dari masing-masing objek didefinisikan di dalam blok `impl` (*implementation*). Hal ini mencerminkan prinsip enkapsulasi di mana fungsi-fungsi yang berkaitan dengan data tertentu dibungkus di dalam objek tersebut:

```
1 struct LahanSawah {  
2     level_air_1: f32,  
3     kelembapan_1: f32,  
4     // ... sensor lainnya ...  
5 }  
6  
7 impl LahanSawah {  
8     // Behavior untuk mengolah data internal sensor  
9     fn hitung_rata_rata_level(&self) -> f32 {  
10         (self.level_air_1 + self.level_air_2 + self.level_air_3) /  
11         3.0  
12     }  
13 }
```

Listing 2: Implementasi OOP pada LahanSawah

Pemisahan antara struktur data (*data fields*) dan implementasi fungsi (*methods*) membuat kode menjadi lebih modular, mudah dipelihara (*maintainable*), dan sangat representatif terhadap sistem instrumentasi di dunia nyata di mana setiap perangkat memiliki spesifikasi data dan fungsi kerja masing-masing.

6 Implementasi Komputasi Numerik

Sesuai dengan kriteria penilaian yang menitikberatkan pada aspek komputasi, sistem ini menerapkan metode **Rata-rata Spasial** (*Spatial Average*) sebagai bentuk pengolahan data numerik sederhana namun efektif. Metode ini digunakan untuk

memitigasi anomali pembacaan sensor pada satu titik tertentu (*outlier*) dan memberikan gambaran kondisi rata-rata seluruh petak sawah.

6.1 Formulasi Matematis

Sistem melakukan komputasi terhadap dua variabel proses yang berbeda. Perhitungan rata-rata dilakukan secara diskrit terhadap tiga titik pengambilan data sensor yang berbeda lokasi.

1. **Rata-rata Level Air ($\bar{L}$):** Dihitung berdasarkan tinggi genangan air pada Titik 1 (L_1), Titik 2 (L_2), dan Titik 3 (L_3) menggunakan persamaan:

$$\bar{L} = \frac{L_1 + L_2 + L_3}{3} \quad (1)$$

2. **Rata-rata Kelembapan Tanah ($\bar{K}$):** Dihitung berdasarkan persentase kelembapan pada Titik 1 (K_1), Titik 2 (K_2), dan Titik 3 (K_3) menggunakan persamaan:

$$\bar{K} = \frac{K_1 + K_2 + K_3}{3} \quad (2)$$

6.2 Integrasi pada Algoritma

Di dalam program Rust, hasil perhitungan $\bar{L}$ dan $\bar{K}$ disimpan dalam variabel bertipe data *floating point* (f32) untuk menjaga akurasi desimal. Nilai-nilai rata-rata inilah yang kemudian dikirimkan menuju unit *controller* untuk dibandingkan dengan *Set Point*. Pendekatan ini memastikan bahwa keputusan aktuator (pompa) didasarkan pada kecenderungan kondisi lahan secara kolektif, bukan hanya berdasarkan pembacaan lokal di satu sudut sawah.

7 Hasil Pengujian

Pengujian sistem dilakukan dengan menjalankan program berbasis terminal (*console application*) menggunakan perintah `cargo run`. Data pembacaan sensor disimulasikan melalui input pengguna secara langsung pada terminal. Pengujian dibagi menjadi tiga skenario utama untuk memvalidasi respon aktuator terhadap berbagai kondisi lahan.

7.1 Skenario 1: Kondisi Genangan Ideal

Pada skenario ini, sistem diberikan input di mana lahan masih memiliki genangan air yang cukup tinggi (di atas 3 cm).

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Finished "dev" profile [unoptimized + debuginfo] target(s) in 0.01s
Running "target/debug/vets_alprog.exe"

=====
SISTEM MONITORING & KONTROL IRIGASI OTOMATIS
=====

--- Masukkan Data Pembacaan Sensor Terbaru ---
Level Air Titik 1 (cm) : 4
Kelembapan Titik 1 (%) : 65
Level Air Titik 2 (cm) : 4
Kelembapan Titik 2 (%) : 70
Level Air Titik 3 (cm) : 5
Kelembapan Titik 3 (%) : 68

=== HASIL EVALUASI SISTEM ===
Rata-rata Level Air : 4.3 cm
Rata-rata Kelembapan : 67.7 %
Keputusan Aktuator : Genangan Air Ideal. Pompa Irigasi -> OFF
=====
```

Figure 2: Hasil Pengujian: Genangan Air Ideal

Berdasarkan hasil eksekusi, program berhasil menghitung rata-rata level air dan menginstruksikan Pompa Irigasi untuk **OFF**.

7.2 Skenario 2: Kondisi Air Surut, Tanah Basah

Pada skenario ini, genangan air disimulasikan berada di bawah batas ideal (< 3 cm), namun kelembapan tanah masih di atas batas kritis ($\geq 65\%$).

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Finished "dev" profile [unoptimized + debuginfo] target(s) in 0.01s
Running "target/debug/vets_alprog.exe"

=====
SISTEM MONITORING & KONTROL IRIGASI OTOMATIS
=====

--- Masukkan Data Pembacaan Sensor Terbaru ---
Level Air Titik 1 (cm) : 2
Kelembapan Titik 1 (%) : 70
Level Air Titik 2 (cm) : 2
Kelembapan Titik 2 (%) : 68
Level Air Titik 3 (cm) : 2
Kelembapan Titik 3 (%) : 69

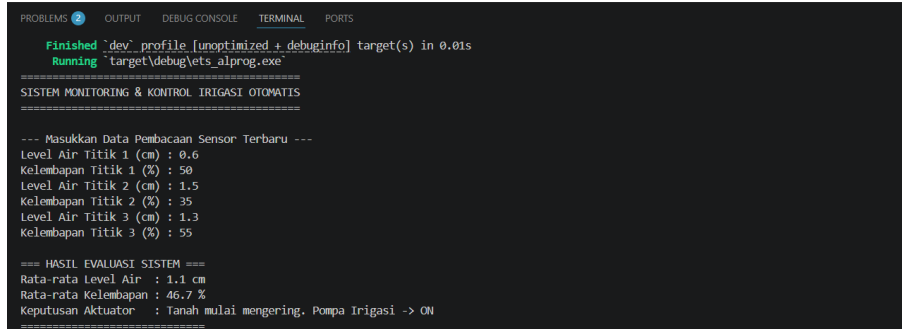
=== HASIL EVALUASI SISTEM ===
Rata-rata Level Air : 2.0 cm
Rata-rata Kelembapan : 69.0 %
Keputusan Aktuator : Air surut, tanah masih basah. Pompa Irigasi -> STANDBY
=====
```

Figure 3: Hasil Pengujian: Air Surut, Pompa Standby

Sistem merespon dengan benar dengan tidak membuang energi secara sia-sia, dan menginstruksikan Pompa Irigasi dalam status **STANDBY**.

7.3 Skenario 3: Kondisi Kekeringan Ekstrem

Skenario ketiga menguji fungsi alarm sistem. Input yang diberikan berupa level air sangat rendah (< 1 cm) dan kelembapan tanah yang mengering ekstrem ($< 65\%$).



```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.01s
Running `target\debug\ets_alprog.exe`

=====
SISTEM MONITORING & KONTROL IRIGASI OTOMATIS
=====

--- Masukkan Data Pembacaan Sensor Terbaru ---
Level Air Titik 1 (cm) : 0.6
Kelembapan Titik 1 (%) : 50
Level Air Titik 2 (cm) : 1.5
Kelembapan Titik 2 (%) : 35
Level Air Titik 3 (cm) : 1.3
Kelembapan Titik 3 (%) : 55

=== HASIL EVALUASI SISTEM ===
Rata-rata Level Air : 1.1 cm
Rata-rata Kelembapan : 46.7 %
Keputusan Aktuator : Tanah mulai mengering. Pompa Irigasi -> ON
=====
```

Figure 4: Hasil Pengujian: Kekeringan Ekstrem

Sistem secara presisi menampilkan peringatan alarm ekstrem dan menginstruksikan Pompa Irigasi untuk **ON (Aliran Maksimal)**.

8 Analisis Hasil

Berdasarkan serangkaian simulasi pengujian yang telah dipaparkan pada Bab 8, kinerja sistem kontrol irigasi multi-variabel ini dapat dianalisis dari beberapa aspek keteknikan instrumentasi:

8.1 Kesesuaian Logika *Controller* dan Efisiensi

Logika percabangan berjenjang (*nested if-else*) yang ditanamkan pada objek *ControllerIrigasi* terbukti mampu mengeksekusi keputusan secara presisi sesuai algoritma. Sistem tidak mengalami tumpang tindih (*overlap*) instruksi meskipun mengevaluasi dua parameter fisis secara bersamaan. Pendekatan memprioritaskan pembacaan level air terbukti efektif menekan aktivitas aktuator yang tidak perlu. Hal ini terlihat pada Skenario 2, di mana pompa tetap berstatus *Standby* saat air surut karena sistem mengenali bahwa residu kelembapan tanah masih memenuhi *Set Point* ($\geq 65\%$). Karakteristik ini sangat krusial untuk menjaga efisiensi penggunaan energi listrik pada sistem irigasi skala besar.

8.2 Keandalan Pengolahan Data (*Spatial Average*)

Implementasi komputasi numerik berupa *Spatial Average* menunjukkan keandalan tinggi dalam meredam efek anomali (*noise*) atau kegagalan pembacaan lokal. Dalam aplikasi persawahan nyata, satu sensor mungkin saja tertutup lumpur tebal atau terinjak sehingga menghasilkan pembacaan ekstrem. Melalui komputasi rata-rata dari tiga titik lokasi yang berbeda, sistem mampu melunakkan efek penyimpangan tersebut sehingga *controller* tidak memicu alarm palsu (*false alarm*).

8.3 Ketahanan Program (*Robustness*)

Dari segi *software engineering*, penggunaan bahasa pemrograman Rust memberikan keunggulan komparatif berupa stabilitas operasional. Fitur validasi input menggunakan teknik *pattern matching* secara konsisten berhasil menyaring data yang korup

(seperti karakter huruf atau *string* kosong) tanpa menyebabkan program mengalami *crash* atau penghentian paksa. Selain itu, eksekusi dalam *infinite loop* berjalan lancar tanpa indikasi kebocoran memori (*memory leak*), yang merepresentasikan kesiapan kode ini jika di-*deploy* ke dalam mikrokontroler *embedded system*.

9 Kesimpulan

Berdasarkan proses pengembangan, komputasi, dan pengujian yang telah dilakukan, dapat ditarik kesimpulan sebagai berikut:

1. Program simulasi instrumentasi kontrol irigasi otomatis berbasis *console* berhasil diimplementasikan menggunakan bahasa pemrograman Rust secara utuh.
2. Paradigma Pemrograman Berbasis Objek (OOP) memanfaatkan **struct** dan **impl** sangat representatif untuk memodelkan sistem fisik, memisahkan entitas *Plant* (lahan sawah dan sensor) dengan unit *Controller*.
3. Komputasi numerik berupa rata-rata spasial (*spatial average*) dari enam pembacaan sensor memberikan *feedback* yang lebih valid dan mewakili kondisi riil lahan persawahan yang luas.
4. Sistem logika berjenjang (*nested if*) berhasil merespons dinamika lahan persawahan dengan memberikan instruksi aktuator yang tepat: **OFF** (aman), **STANDBY** (siaga), dan **ON** (kritis).